

NOM Prénom

Programmation Ingénierie des Médias 1 (ProgIM1)**Examen d'entraînement - Partie 2**

24.01.2026

Durée : 90 minutes

	Points obtenus	Points totaux
Question 1		7
Question 2		8
Question 3		8
Question 4		6
Question 5		5
Question 6		6
Total		40
Note finale		

Vous avez le droit à une page recto-verso de résumé avec vos propres notes.

Vous pouvez répondre aux questions en français ou en anglais.

Toute tentative de triche sera sanctionnée par la note de 1.

Merci de mettre votre prénom et nom sur chaque page.

Examen d'entraînement préparé avec l'aide de Ludovic Delafontaine, Hadrien Louis, GitHub Copilot et Claude (Anthropic).

Page laissée vide

1. Analyse de code : Tableaux et boucles

/ 7 pts

Le code Java ci-dessous doit calculer la moyenne des notes d'un groupe de personnes qui étudient et afficher combien de notes sont supérieures ou égales à 4 (la note de passage).

Ce code contient **exactement 4 erreurs**.

Pour chaque erreur :

1. Indiquez la/les ligne(s) concernée(s)
2. Expliquez ce qui ne fonctionne pas et pourquoi
3. Proposez une correction

```
1 public class AnalyseNotes {
2     public static void main(String[] args) {
3         double[] notes = {5.5, 3.0, 4.5, 2.5, 6.0, 4.0};
4
5         double somme = 0;
6         int nombreReussites = 0;
7
8         // Calcul de la somme des notes
9         for (int i = 0; i <= notes.length; i++) {
10             somme = somme + notes[i];
11         }
12
13         // Calcul de la moyenne
14         double moyenne = somme / notes.size();
15
16         // Comptage des réussites
17         for (int i = 1; i < notes.length; i++) {
18             if (notes[i] > 4) {
19                 nombreReussites++;
20             }
21         }
22
23         System.out.println("Moyenne : " + moyenne);
24         System.out.println("Nombre de réussites : " + nombreReussites);
25     }
26 }
```

Les 4 erreurs (1.75 point par erreur identifiée et corrigée) :	Points
1. Ligne 9 : <code>i <= notes.length</code> → doit être <code>i < notes.length</code> • Problème : Provoque un <code>ArrayIndexOutOfBoundsException</code> car les indices vont de 0 à <code>length-1</code> • Correction : <code>for (int i = 0; i < notes.length; i++)</code> 2. Ligne 14 : <code>notes.size()</code> → doit être <code>notes.length</code> • Problème : Les tableaux utilisent <code>.length</code> (attribut), pas <code>.size()</code> (méthode des collections) • Correction : <code>double moyenne = somme / notes.length;</code> 3. Ligne 17 : <code>i = 1</code> → doit être <code>i = 0</code> • Problème : On ignore la première note (index 0), ce qui fausse le comptage • Correction : <code>for (int i = 0; i < notes.length; i++)</code> 4. Ligne 18 : <code>notes[i] > 4</code> → doit être <code>notes[i] >= 4</code> • Problème : On n'inclut pas les notes exactement égales à 4.0 (note de passage) • Correction : <code>if (notes[i] >= 4)</code> (Note : Il n'y a en fait que 4 erreurs principales. Si des erreurs logiques supplémentaires sont trouvées, les accepter si elles sont justifiées) Code corrigé complet : <pre>public class AnalyseNotes { public static void main(String[] args) { double[] notes = {5.5, 3.0, 4.5, 2.5, 6.0, 4.0}; double somme = 0; int nombreReussites = 0; for (int i = 0; i < notes.length; i++) { somme = somme + notes[i]; } double moyenne = somme / notes.length; for (int i = 0; i < notes.length; i++) { if (notes[i] >= 4) { nombreReussites++; } } System.out.println("Moyenne : " + moyenne); System.out.println("Nombre de réussites : " + nombreReussites); } }</pre>	

2. Lecture et complétion de fonctions

/ 8 pts

Partie A (4 points) : Lisez la fonction suivante et expliquez ce qu'elle fait :

```
public static int mystere(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    return n * mystere(n - 1);  
}
```

1. Que retourne `mystere(5)` ? **(1 point)**
2. Quel est le nom mathématique de cette opération ? **(1 point)**
3. Cette fonction est-elle récursive ou itérative ? Justifiez. **(2 points)**

Partie B (4 points) : Complétez la fonction suivante qui trouve le maximum dans un tableau :

```
public static int trouverMaximum(int[] tableau) {  
    int max = tableau[0];  
  
    for (int i = ____; i < ____; i++) {  
        if (tableau[i] ____ max) {  
            max = ____;  
        }  
    }  
  
    return max;  
}
```

Partie A : Lecture de code	Points
1. <code>mystere(5)</code> retourne 120 (1 point) • Calcul : $5 \times 4 \times 3 \times 2 \times 1 = 120$ 2. C'est une factorielle (1 point) 3. Cette fonction est récursive (2 points) car : • Elle s'appelle elle-même (<code>mystere(n - 1)</code>) • Elle a un cas de base (<code>n <= 1</code>) qui arrête la récursion (1 point pour "récursive", 1 point pour la justification correcte)	

Partie B : Complétion de code	Points
<pre>public static int trouverMaximum(int[] tableau) { int max = tableau[0]; for (int i = 1; i < tableau.length; i++) { // (2 points) if (tableau[i] > max) { // (1 point) max = tableau[i]; // (1 point) } } return max; }</pre>	

3. Algorithme de recherche dans un tableau

/ 8 pts

Complétez la fonction Java suivante qui cherche si un élément existe dans un tableau et retourne son index (position). Si l'élément n'existe pas, la fonction retourne -1.

```
public class RechercheTableau {  
    /**  
     * Cherche un élément dans un tableau  
     * @param tableau Le tableau dans lequel chercher  
     * @param element L'élément à chercher  
     * @return L'index de l'élément, ou -1 s'il n'existe pas  
     */  
    public static int rechercher(int[] tableau, int element) {  
        // TODO: Implémenter la recherche  
  
    }  
  
    public static void main(String[] args) {  
        int[] nombres = {10, 25, 7, 42, 18, 3};  
  
        System.out.println(rechercher(nombres, 42)); // Doit afficher 3  
        System.out.println(rechercher(nombres, 99)); // Doit afficher -1  
    }  
}
```

```
public static int rechercher(int[] tableau, int element) {  
    // Parcourir le tableau (2 points - boucle correcte)  
    for (int i = 0; i < tableau.length; i++) {  
        // Comparer chaque élément (2 points)  
        if (tableau[i] == element) {  
            // Retourner l'index si trouvé (2 points)  
            return i;  
        }  
    }  
  
    // Retourner -1 si pas trouvé (2 points)  
    return -1;  
}
```

Points

Répartition des points :

- Boucle correcte parcourant tout le tableau : 2 points
- Comparaison de chaque élément : 2 points
- Retour de l'index quand trouvé : 2 points
- Retour de -1 si non trouvé : 2 points

Total : 8 points

Variantes acceptables :

- Utilisation de for-each (si adaptation correcte)
- Utilisation de while

4. Parcours avancés de tableaux

/ 6 pts

Pour chacune des trois tâches suivantes, complétez **uniquement la ligne de la boucle for**.

Partie A (2 points) : Complétez la boucle pour parcourir le tableau **en sens inverse** :

```
int[] nombres = {10, 20, 30, 40, 50};

for (int i = __; __; __) {
    System.out.println(nombres[i]);
}
```

Partie B (2 points) : Complétez la boucle pour parcourir en **ignorant le premier et le dernier élément** :

```
int[] valeurs = {5, 12, 8, 15, 3};

for (int i = __; __; __) {
    System.out.println(valeurs[i]);
}
```

Partie C (2 points) : Complétez la boucle pour parcourir uniquement les éléments aux **indices pairs** (0, 2, 4, ...):

```
String[] mots = {"chat", "chien", "oiseau", "poisson", "souris"};

for (int i = __; __; __) {
    System.out.println(mots[i]);
}
```

Partie A : Parcours inversé (2 points)	Points
<pre>for (int i = nombres.length - 1; i >= 0; i--) { System.out.println(nombres[i]); }</pre> (0.7 pt initialisation, 0.7 pt condition, 0.6 pt décrément)	

Partie B : Sans premier ni dernier (2 points)	Points
<pre>for (int i = 1; i < valeurs.length - 1; i++) { System.out.println(valeurs[i]); }</pre> (0.7 pt pour i = 1, 0.7 pt pour length - 1, 0.6 pt pour i++)	

Partie C : Indices pairs (2 points)	Points
<pre>for (int i = 0; i < mots.length; i += 2) { System.out.println(mots[i]); }</pre> (0.5 pt pour i = 0, 0.5 pt pour i < length, 1 pt pour i += 2) Alternative acceptable : <pre>for (int i = 0; i < mots.length; i++) { if (i % 2 == 0) { System.out.println(mots[i]); } }</pre>	

<pre> } }</pre> (mais moins efficace, accepter avec 1.5 point)	
---	--

5. Scanner et interaction avec l'utilisatrice

/ 5 pts

Complétez le code Java suivant pour créer un petit programme interactif qui :

1. Demande le prénom de l'utilisatrice
2. Demande son âge
3. Affiche un message de bienvenue personnalisé

```
import java.util.Scanner;

public class Bienvenue {
    public static void main(String[] args) {
        // TODO: Créer un objet Scanner

        System.out.print("Entrez votre prénom : ");
        // TODO: Lire le prénom

        System.out.print("Entrez votre âge : ");
        // TODO: Lire l'âge (nombre entier)

        // TODO: Afficher "Bonjour [prénom], vous avez [âge] ans !"

        // TODO: Fermer le Scanner
    }
}
```

<pre>import java.util.Scanner; public class Bienvenue { public static void main(String[] args) { // Créer un objet Scanner (1 point) Scanner scanner = new Scanner(System.in); System.out.print("Entrez votre prénom : "); // Lire le prénom (1 point) String prenom = scanner.nextLine(); System.out.print("Entrez votre âge : "); // Lire l'âge (1 point) int age = scanner.nextInt(); // Afficher le message (1 point) System.out.println("Bonjour " + prenom + ", vous avez " + age + " ans !"); // Fermer le Scanner (1 point) scanner.close(); } }</pre>	Points
Total : 5 points (1 point par TODO complété correctement)	

6. Manipulation de chaînes de caractères (Strings)

/ 6 pts

Complétez le code suivant qui manipule des chaînes de caractères :

```
public class ManipulationStrings {
    public static void main(String[] args) {
        String texte = "Programmation en Java";

        // TODO: Afficher la longueur de la chaîne
        System.out.println("Longueur : " + ____);

        // TODO: Afficher le texte en majuscules
        System.out.println("Majuscules : " + ____);

        // TODO: Extraire le mot "Java" (indices 17 à 21)
        String mot = texte.__(17, 21);
        System.out.println("Mot extrait : " + mot);

        // TODO: Vérifier si le texte contient "Java"
        if (texte.__(__)) {
            System.out.println("Le texte contient Java !");
        }
    }
}
```

Output attendu :

Longueur : 21
Majuscules : PROGRAMMATION EN JAVA
Mot extrait : Java
Le texte contient Java !

<pre>public class ManipulationStrings { public static void main(String[] args) { String texte = "Programmation en Java"; // Afficher la longueur (1.5 point) System.out.println("Longueur : " + texte.length()); // Afficher en majuscules (1.5 point) System.out.println("Majuscules : " + texte.toUpperCase()); // Extraire "Java" (1.5 point) String mot = texte.substring(17, 21); System.out.println("Mot extrait : " + mot); // Vérifier si contient "Java" (1.5 point) if (texte.contains("Java")) { System.out.println("Le texte contient Java !"); } } }</pre>	Points
---	--------